

Отчёт по практической работе

Разработка ORM-слоя веб-форума с использованием Sequelize

Этап 2. Разработка связывающего кода

Выполнил: студент гр. 327

Хворов Андрей Максимович

Содержание

1	Введение	2
2	Структура проекта	3
3	Схема базы данных	3
4	Конфигурация ORM	4
5	Модели (entity-классы)	4
5.1	Модель User	4
5.2	Модель Section	5
5.3	Модель Topic	5
5.4	Модель Message	5
5.5	Модель Attachment	5
5.6	Ассоциации	5
6	DAO-классы	6
6.1	UserDao	6
6.2	SectionDao	7
6.3	TopicDao	7
6.4	MessageDao	8
6.5	AttachmentDao	8
7	Модульные тесты	9
7.1	Общий подход	9
7.2	Тесты UserDao (24 теста)	9
7.3	Тесты SectionDao (10 тестов)	11
7.4	Тесты TopicDao (12 тестов)	12
7.5	Тесты MessageDao (18 тестов)	13
7.6	Тесты AttachmentDao (8 тестов)	14
8	Результаты тестирования	15
9	Файл сборки	15
10	Заключение	16

1 Введение

Целью данной работы является реализация ORM-слоя для базы данных веб-форума, спроектированной на первом этапе. В рамках задания необходимо:

- определить классы хранимых объектов (entity-модели), соответствующие таблицам базы данных;
- задать отображение полей моделей на столбцы таблиц и определить ассоциации между моделями;
- разработать DAO-классы (Data Access Object), реализующие операции доступа к данным;
- написать модульные тесты, покрывающие все методы DAO-классов с нетривиальной логикой, включая все варианты поведения каждого метода;
- подготовить конфигурацию ORM и файл сборки проекта.

Исходное задание предполагает стек Java/Hibernate/TestNG/Ant. В данной работе используется эквивалентный стек на платформе Node.js (таблица 1).

Таблица 1: Соответствие технологических стеков

Оригинальный стек	Используемый стек
Java	JavaScript (Node.js)
Hibernate ORM	Sequelize ORM 6
hibernate.cfg.xml	src/config/database.js
@Entity-классы	Sequelize-модели (define)
DAO-классы	DAO-классы (тот же паттерн)
TestNG	Jest 29
Ant build.xml	npm scripts (package.json)
PostgreSQL	PostgreSQL (dev) / SQLite (test)

2 Структура проекта

Проект организован следующим образом:

- `src/config/database.js` — конфигурация подключения к БД (аналог `hibernate.cfg.xml`);
- `src/models/` — пять файлов моделей (`User`, `Section`, `Topic`, `Message`, `Attachment`) и файл `index.js`, инициализирующий экземпляр `Sequelize` и задающий ассоциации;
- `src/dao/` — пять DAO-классов (по одному на каждую сущность);
- `tests/` — файл подготовки тестовых данных (`setup.js`) и пять файлов тестов;
- `package.json` — файл сборки (аналог `build.xml`), описывающий зависимости и скрипты запуска;
- `jest.config.js` — конфигурация тестового фреймворка.

Результаты работы помещены в SVN-репозиторий студента.

3 Схема базы данных

База данных состоит из пяти таблиц: `user`, `section`, `topic`, `message`, `attachment`. Все внешние ключи имеют ограничение `ON DELETE CASCADE`. Связи между таблицами:

- `user` → `section`: один пользователь может создать много разделов;
- `user` → `topic`: один пользователь может создать много тем;
- `user` → `message`: один пользователь может написать много сообщений;
- `section` → `topic`: раздел содержит много тем;
- `topic` → `message`: тема содержит много сообщений;

- `message` → `attachment`: сообщение может иметь несколько вложений.

DDL-скрипт создания таблиц приведён в приложенном файле `create_db.sql`.

4 Конфигурация ORM

Конфигурация определена в файле `src/config/database.js` и выбирается в зависимости от переменной окружения `NODE_ENV`.

Для рабочей среды (`development`) используется PostgreSQL с параметрами подключения, задаваемыми через переменные окружения (хост, порт, имя базы, логин, пароль).

Для тестовой среды (`test`) используется SQLite с хранением в оперативной памяти (`:memory:`). Это обеспечивает быстрый прогон тестов без необходимости запущенного сервера PostgreSQL и гарантирует полную изоляцию тестов.

В обоих случаях включена опция `underscored: true`, обеспечивающая автоматическое преобразование camelCase-имён полей модели в snake_case-имена столбцов таблиц. Опция `timestamps: false` отключает автоматическое добавление полей `createdAt/updatedAt` средствами Sequelize, поскольку временные метки управляются вручную.

5 Модели (entity-классы)

Каждая модель определяется функцией, принимающей экземпляр Sequelize и вызывающей `sequelize.define()` — аналог аннотации `@Entity` в Hibernate. Для каждой модели явно указано имя таблицы (`tableName`).

5.1 Модель User

Поля: `userId` (ПК, автоинкремент), `login` (строка до 64 символов, уникальный, обязательный), `password` (строка до 255 символов, обязательный), `registeredAt` (дата, по умолчанию NOW), `role` (строка, допустимые значения `user` и `moderator`, по умолчанию `user`), `isBlocked` (булево, по умолчанию `false`).

Валидация роли реализована через `validate.isIn`.

5.2 Модель Section

Поля: `sectionId` (PK), `title` (строка до 128 символов, обязательный), `description` (текст, необязательный), `userId` (FK на User, обязательный), `createdAt` (дата, по умолчанию NOW).

5.3 Модель Topic

Поля: `topicId` (PK), `sectionId` (FK на Section), `userId` (FK на User), `title` (строка до 255 символов, обязательный), `createdAt` (дата).

5.4 Модель Message

Поля: `messageId` (PK), `topicId` (FK на Topic), `userId` (FK на User), `title` (строка, необязательный), `body` (текст, обязательный), `postedAt` (дата).

5.5 Модель Attachment

Поля: `attachmentId` (PK), `messageId` (FK на Message), `filename` (строка до 255 символов), `filepath` (строка до 512 символов), `filesize` (целое число), `mediaType` (строка, необязательный), `uploadedAt` (дата).

5.6 Ассоциации

Ассоциации задаются в файле `src/models/index.js` с помощью методов `hasMany` и `belongsTo` — аналог аннотаций `@OneToMany`/`@ManyToOne` в Hibernate. Для всех связей указан `onDelete: CASCADE`. Псевдонимы (alias) позволяют обращаться к связанным сущностям по осмысленным именам: `creator` (автор раздела), `author` (автор темы или сообщения), `attachments` (вложения сообщения) и т. д.

6 DAO-классы

Каждый DAO-класс инкапсулирует логику доступа к данным соответствующей сущности и экспортирует единственный экземпляр (паттерн Singleton). Все методы являются асинхронными (**async**) и возвращают промисы.

6.1 UserDao

- `createUser(login, password, role)` — создание пользователя. Роль по умолчанию `user`. При дублировании логина или невалидной роли выбрасывается исключение.
- `findByLogin(login)` — поиск по логину. Возвращает объект пользователя или `null`.
- `findById(id)` — поиск по первичному ключу. Возвращает объект или `null`.
- `findAll(filters)` — получение списка пользователей с необязательной фильтрацией по полям `role` и `isBlocked`. Результат отсортирован по дате регистрации.
- `updateRole(userId, newRole)` — изменение роли. Находит пользователя, обновляет поле, сохраняет. Возвращает обновлённый объект или `null`, если пользователь не найден. При невалидной роли — исключение.
- `setBlocked(userId, isBlocked)` — блокировка и разблокировка. Аналогично `updateRole`.
- `updatePassword(userId, newPassword)` — смена пароля. Возвращает обновлённый объект или `null`.
- `getUserStats(userId)` — агрегированная статистика: количество сообщений и тем пользователя. Два счётчика запрашиваются параллельно через `Promise.all`. Возвращает объект `{userId, login, messageCount, topicCount}` или `null`.

6.2 SectionDao

- `createSection(title, description, userId)` — создание раздела. Описание может быть `null`. При пустом заголовке — исключение.
- `findAll()` — получение всех разделов с обогащёнными данными: информация о создателе (`join`), количество тем (подсчёт) и последнее сообщение в разделе (поиск по связанным темам). Используется N+1-подход, допустимый при небольшом числе разделов.
- `findById(sectionId)` — поиск раздела с информацией о создателе. Возвращает объект или `null`.
- `findByTitle(title)` — поиск по названию. Возвращает объект или `null`.
- `deleteSection(sectionId)` — удаление раздела. Благодаря каскадному удалению вместе с разделом удаляются все его темы, сообщения и вложения. Возвращает количество удалённых записей (0 или 1).

6.3 TopicDao

- `createTopic(sectionId, userId, title)` — создание темы.
- `findBySectionId(sectionId, options)` — получение тем раздела с данными автора. Поддерживает пагинацию (`limit, offset`) и настраиваемый порядок сортировки (по умолчанию — по дате создания по убыванию).
- `findById(topicId)` — поиск темы с данными автора. Возвращает объект или `null`.
- `deleteTopic(topicId)` — удаление темы с каскадным удалением сообщений и вложений. Возвращает 0 или 1.
- `getTopicsByUser(userId)` — все темы, созданные пользователем, отсортированные по дате создания по убыванию. Возвращает массив (может быть пустым).
- `countBySectionId(sectionId)` — количество тем в разделе.

6.4 MessageDao

- `createMessage(topicId, userId, body, title)` — создание сообщения. Заголовок необязателен (по умолчанию `null`).
- `findByTopicId(topicId, options)` — получение сообщений темы с данными автора и списком вложений (`eager loading`). Поддерживает пагинацию. Результат отсортирован по дате публикации по возрастанию.
- `deleteMessage(messageId)` — удаление сообщения с каскадным удалением вложений. Возвращает 0 или 1.
- `countByUserId(userId)` — количество сообщений пользователя.
- `getLastMessageInSection(sectionId)` — последнее (самое свежее) сообщение в разделе, определяемое через `join` с темами данного раздела. Включает данные автора. Возвращает объект или `null`.
- `getActivityStats(dateFrom, dateTo, sectionId, userId)` — статистика активности: количество сообщений по дням за указанный период. Поддерживает необязательные фильтры по разделу и пользователю. Группировка выполняется SQL-функцией `DATE()`. Возвращает массив объектов `{date, count}`.

6.5 AttachmentDao

- `createAttachment(messageId, filename, filepath, filesize, mediaType)` — создание вложения. Тип медиа необязателен.
- `findByMessageId(messageId)` — все вложения сообщения, отсортированные по дате загрузки. Возвращает массив (может быть пустым).
- `findById(attachmentId)` — поиск вложения по идентификатору. Возвращает объект или `null`.
- `deleteByMessageId(messageId)` — удаление всех вложений сообщения. Возвращает количество удалённых записей.

7 Модульные тесты

7.1 Общий подход

Тесты написаны с использованием Jest (аналог TestNG). Перед каждым тестом (`beforeEach`) выполняется полное пересоздание схемы БД (`sync({force: true})`) и заполнение тестовыми данными. Это гарантирует полную изоляцию тестов друг от друга: каждый тест работает на одинаковом начальном наборе данных.

Тестовые данные воспроизводят скрипт `fill_db.sql` и включают 4 пользователя (2 модератора и 2 обычных), 3 раздела, 4 темы, 5 сообщений и 3 вложения.

Тесты проверяют все методы DAO-классов с нетривиальной логикой. Для каждого метода, имеющего несколько вариантов поведения (нашёл результат / не нашёл, корректные данные / ошибка валидации), написаны отдельные тесты на каждый вариант. Во всех тестах проверяются все значимые свойства полученного результата с помощью соответствующих `assert`-методов (`expect`).

Всего в проекте 72 теста, сгруппированных в 5 файлов.

7.2 Тесты UserDao (24 теста)

Метод	Проверяемые сценарии
<code>createUser</code>	1) Создание с корректными данными — проверяются все поля: <code>userId</code>, <code>login</code>, <code>password</code>, <code>role</code> (значение по умолчанию), <code>isBlocked</code>, <code>registeredAt</code>. 2) Создание модератора при явном указании роли — проверяется <code>role='moderator'</code>. 3) Дубликат логина — проверяется, что выбрасывается исключение. 4) Невалидная роль — проверяется исключение.

Метод	Проверяемые сценарии
<code>findByLogin</code>	1) Пользователь найден — проверяются <code>userId</code>, <code>login</code>, <code>password</code>, <code>role</code>, <code>isBlocked</code>. 2) Пользователь не найден — проверяется возврат <code>null</code>.
<code>findById</code>	1) Пользователь найден — проверяются <code>userId</code>, <code>login</code>, <code>role</code>. 2) Несуществующий <code>id</code> — возврат <code>null</code>.
<code>findAll</code>	1) Без фильтров — проверяется длина массива (4), порядок сортировки (первый и последний элементы). 2) Фильтр по роли — проверяется, что возвращены только модераторы (2 записи), проверяются логины. 3) Фильтр по блокировке — после блокировки пользователя проверяется фильтр <code>isBlocked=true</code> (1 запись) и <code>isBlocked=false</code> (3 записи).
<code>updateRole</code>	1) Смена <code>user</code> → <code>moderator</code> — проверяется поле <code>role</code> и его сохранение в БД (повторный запрос). 2) Смена <code>moderator</code> → <code>user</code> — проверяется <code>role</code>. 3) Несуществующий пользователь — проверяется <code>null</code>. 4) Невалидная роль — проверяется исключение.
<code>setBlocked</code>	1) Блокировка — проверяется <code>isBlocked=true</code> и сохранение в БД. 2) Разблокировка — блокировка, затем разблокировка, проверяется <code>isBlocked=false</code>. 3) Несуществующий пользователь — <code>null</code>.

Метод	Проверяемые сценарии
<code>updatePassword</code>	1) Успешное обновление — проверяется новый пароль и его сохранение в БД. 2) Несуществующий <code>id</code> — <code>null</code> .
<code>getUserStats</code>	1) Статистика существующего пользователя — проверяются <code>userId</code> , <code>login</code> , <code>messageCount=2</code> , <code>topicCount=1</code> . 2) Нулевая статистика нового пользователя — создаётся пользователь без активности, проверяются нулевые счётчики. 3) Несуществующий пользователь — <code>null</code> .

7.3 Тесты SectionDao (10 тестов)

Метод	Проверяемые сценарии
<code>createSection</code>	1) Создание с полными данными — проверяются <code>sectionId</code> , <code>title</code> , <code>description</code> , <code>userId</code> , <code>createdAt</code> . 2) Создание без описания (<code>null</code>) — проверяется <code>description=null</code> . 3) Пустой заголовок — проверяется исключение.
<code>findAll</code>	1) Проверяется список всех разделов (3 записи). Для каждого раздела проверяются: название, описание, данные создателя (<code>creator.login</code>), количество тем (<code>topicCount</code>), наличие и содержимое последнего сообщения (<code>lastMessage.title</code> или <code>lastMessage.body</code>).

Метод	Проверяемые сценарии
<code>findById</code>	1) Раздел найден — проверяются <code>sectionId</code> , <code>title</code> , наличие и содержимое <code>creator</code> . 2) Несуществующий <code>id</code> — <code>null</code> .
<code>findByTitle</code>	1) Раздел найден — проверяются <code>sectionId</code> , <code>title</code> . 2) Несуществующее название — <code>null</code> .
<code>deleteSection</code>	1) Успешное удаление — проверяется возврат <code>1</code> , повторный поиск возвращает <code>null</code> . 2) Каскадное удаление — после удаления раздела проверяется, что его темы и сообщения также удалены (запросы к таблицам <code>topic</code> и <code>message</code>). 3) Несуществующий раздел — возврат <code>0</code> .

7.4 Тесты TopicDao (12 тестов)

Метод	Проверяемые сценарии
<code>createTopic</code>	1) Создание — проверяются <code>topicId</code> , <code>sectionId</code> , <code>userId</code> , <code>title</code> , <code>createdAt</code> .
<code>findBySectionId</code>	1) Темы раздела — проверяются количество (2), порядок сортировки (DESC по умолчанию), названия и логины авторов. 2) Пустой раздел — создаётся раздел без тем, проверяется пустой массив. 3) Пагинация — с <code>limit=1</code> : две страницы содержат разные записи. 4) Сортировка по возрастанию — проверяется обратный порядок записей.

Метод	Проверяемые сценарии
<code>findById</code>	1) Тема найдена — проверяются <code>topicId</code> , <code>title</code> , <code>sectionId</code> , <code>author.login</code> . 2) Несуществующий <code>id</code> — <code>null</code> .
<code>deleteTopic</code>	1) Удаление с каскадом — проверяется возврат 1, тема не найдена повторно, сообщения темы удалены. 2) Несуществующая тема — возврат 0.
<code>getTopicsByUser</code>	1) Пользователь с темами — проверяются количество (1), название, <code>userId</code> . 2) Пользователь без тем — пустой массив.
<code>countBySectionId</code>	1) Раздел с темами — проверяется <code>count=2</code> . 2) Пустой раздел — <code>count=0</code> .

7.5 Тесты MessageDao (18 тестов)

Метод	Проверяемые сценарии
<code>createMessage</code>	1) С заголовком — проверяются <code>messageId</code> , <code>topicId</code> , <code>userId</code> , <code>title</code> , <code>body</code> , <code>postedAt</code> . 2) Без заголовка — проверяется <code>title=null</code> .
<code>findByTopicId</code>	1) Сообщения с авторами и вложениями — проверяются количество (2), для каждого: <code>title</code> , <code>author.login</code> , длина массива <code>attachments</code> , <code>filename</code> вложения. 2) Пагинация — две страницы содержат разные сообщения. 3) Пустая тема — пустой массив.

Метод	Проверяемые сценарии
<code>deleteMessage</code>	1) Удаление — возврат 1, повторный запрос подтверждает удаление. 2) Каскадное удаление вложений — после удаления сообщения его вложения отсутствуют. 3) Несуществующее сообщение — возврат 0.
<code>countByUserId</code>	1) Пользователь с сообщениями — <code>count=2</code> . 2) Пользователь без сообщений — <code>count=0</code> .
<code>getLastMessage-InSection</code>	1) Раздел 2 — проверяются <code>messageId</code> , <code>body</code> , <code>author.login</code> последнего сообщения. 2) Другой раздел (3) — аналогичные проверки для второго набора данных. 3) Раздел без сообщений — <code>null</code> .
<code>getActivityStats</code>	1) Общая статистика за период — проверяются количество дней (3), для каждого дня: дата и число сообщений. 2) Фильтр по пользователю — проверяется уменьшенное количество записей и корректные счётчики. 3) Фильтр по разделу — проверяется фильтрация через связанные темы. 4) Даты вне диапазона — пустой массив.

7.6 Тесты AttachmentDao (8 тестов)

Метод	Проверяемые сценарии
<code>createAttachment</code>	1) С типом медиа — проверяются <code>attachmentId</code> , <code>messageId</code> , <code>filename</code> , <code>filepath</code> , <code>filesize</code> , <code>mediaType</code> , <code>uploadedAt</code> . 2) Без типа медиа — проверяется <code>mediaType=null</code> .
<code>findById</code>	1) Сообщение с вложениями — проверяются количество (1), <code>filename</code> , <code>filepath</code> , <code>filesize</code> , <code>mediaType</code> . 2) Сообщение без вложений — пустой массив.
<code>findByMessageId</code>	1) Вложение найдено — проверяются <code>attachmentId</code> , <code>filename</code> , <code>filesize</code> , <code>mediaType</code> . 2) Несуществующий id — <code>null</code> .
<code>deleteByMessageId</code>	1) Удаление всех вложений — возврат числа удалённых, повторный запрос возвращает пустой массив. 2) Сообщение без вложений — возврат 0.

8 Результаты тестирования

Все 72 теста пройдены успешно. Результаты покрытия кода приведены в таблице 7.

Покрытие по строкам и функциям составляет 100%. Общее покрытие ветвлений — 93,87%. Непокрытые ветви относятся к инфраструктурному коду: выбор конфигурации по переменной окружения в `database.js`, проверка диалекта в `setup.js` и одна ветвь тернарного оператора в `SectionDao.js`.

9 Файл сборки

Файл `package.json` выполняет роль `build.xml` (Ant). Он определяет:

Таблица 7: Покрытие кода тестами

Файл	Stmts	Branch	Funcs	Lines
src/config/database.js	100%	91,66%	100%	100%
src/dao/AttachmentDao.js	100%	100%	100%	100%
src/dao/MessageDao.js	100%	100%	100%	100%
src/dao/SectionDao.js	100%	50%	100%	100%
src/dao/TopicDao.js	100%	100%	100%	100%
src/dao/UserDao.js	100%	100%	100%	100%
src/models/*.js	100%	100%	100%	100%
tests/setup.js	100%	50%	100%	100%
Итого	100%	93,87%	100%	100%

- зависимости проекта: `sequelize` (ORM), `pg` и `pg-hstore` (драйвер PostgreSQL);
- зависимости для разработки: `jest` (тестовый фреймворк), `sqlite3` (драйвер для тестовой БД);
- скрипты: `npm test` — запуск тестов, `npm run test:coverage` — запуск с отчётом о покрытии.

Установка зависимостей выполняется командой `npm install`, запуск тестов — командой `npm test`.

10 Заключение

В ходе работы реализован полный ORM-слой для базы данных веб-форума:

1. Определены 5 моделей (entity-классов) с указанием типов данных, ограничений и ассоциаций между ними.
2. Разработаны 5 DAO-классов, покрывающих операции CRUD, фильтрацию, пагинацию, каскадное удаление и агрегатные запросы.
3. Написаны 72 модульных теста, проверяющих все методы всех DAO-классов. Для каждого метода протестированы все варианты поведения (успешный результат, пустой результат, ошибка валидации). Во

всех тестах проверяются все значимые свойства результата. Покрывание кода составляет 100% по строкам и функциям.

4. Подготовлены конфигурационные файлы ORM и сборки проекта.

Использование Node.js/Sequelize обеспечивает функциональную эквивалентность решению на Java/Hibernate при сохранении тех же архитектурных паттернов: entity-модели, DAO-слой, модульные тесты с изолированным окружением.